

MODUL PEMBELAJARAN

LARAVEL 13 UNTUK PEMULA

Studi Kasus:

Sistem Informasi Arsip Dokumen (SIARSIP)

Instalasi • Model & Migration • Relasi Database • CRUD • Upload & Kelola Berkas

Disusun secara sistematis dan terstruktur

PHP 8.2+ • Laravel 13 • MySQL • Bootstrap 5

Jajang Nurjaman, S.Kom.

DAFTAR ISI

BAB 1	Pendahuluan
BAB 2	Instalasi dan Konfigurasi Project
BAB 3	Desain Database: Model dan Migration
BAB 4	Menampilkan Data dari Database
BAB 5	Insert Data (Tambah Arsip Dokumen)
BAB 6	Menampilkan Detail Data dan Unduh Berkas
BAB 7	Edit dan Update Data
BAB 8	Delete Data (Hapus Arsip)
BAB 9	Penyempurnaan: Validasi, Keamanan, dan Akses
BAB 10	Penutup

BAB 1 — PENDAHULUAN

1.1 Tentang Modul Ini

Modul ini disusun sebagai panduan belajar Laravel 13 secara terstruktur, mulai dari instalasi, konsep dasar Model dan Migration, hingga implementasi fitur Create, Read, Update, dan Delete (CRUD) secara lengkap. Berbeda dengan tutorial pada umumnya yang menggunakan studi kasus generik seperti data produk, modul ini disusun langsung dengan studi kasus yang aplikatif, yaitu Sistem Informasi Arsip Dokumen (disingkat SIARSIP), sehingga pembaca dapat langsung memahami penerapan konsep Laravel pada kebutuhan dunia nyata.

Setiap bab disusun dengan pola yang konsisten: penjelasan konsep terlebih dahulu, kemudian langkah praktik, potongan kode (script) lengkap, dan penjelasan baris demi baris atau bagian demi bagian dari kode tersebut. Dengan pola ini, pembaca tidak hanya bisa "copy-paste" kode, tetapi juga memahami alasan dan logika di balik setiap baris yang dituliskan.

1.2 Apa yang Baru di Laravel 13

Laravel 13 melanjutkan arah pengembangan Laravel modern yang berfokus pada kesederhanaan struktur folder, performa, dan developer experience. Beberapa poin penting yang perlu dipahami sebelum memulai:

- Struktur project yang lebih ramping — file konfigurasi seperti Kernel HTTP dan beberapa file bootstrap sudah dipangkas dan disatukan ke dalam bootstrap/app.php.
- Skema routing dan middleware didaftarkan secara terpusat melalui bootstrap/app.php, bukan lagi melalui berkas Kernel.php seperti pada versi Laravel sebelumnya.
- Dukungan penuh terhadap PHP versi terbaru (minimal PHP 8.2) dengan peningkatan performa Eloquent ORM dan query builder.
- Starter kit modern (Laravel Breeze/Jetstream generasi baru) untuk autentikasi yang lebih cepat diimplementasikan.
- Peningkatan pada Artisan CLI, termasuk kemudahan pembuatan Model beserta Migration, Factory, Seeder, dan Controller sekaligus dalam satu perintah.

Untuk kebutuhan modul ini, kita akan banyak memanfaatkan poin ketiga dan kelima di atas, yaitu Artisan CLI untuk mempercepat pembuatan Model, Migration, dan Controller pada studi kasus Sistem Arsip Dokumen.

1.3 Studi Kasus: Sistem Informasi Arsip Dokumen (SIARSIP)

Studi kasus yang digunakan pada modul ini adalah aplikasi pengarsipan dokumen digital. Aplikasi ini digunakan untuk mencatat dan menyimpan dokumen-dokumen penting suatu instansi (misalnya surat masuk, surat keluar, sertifikat, kontrak, atau laporan) secara digital, lengkap dengan kategori dokumen, nomor arsip yang unik, serta berkas fisik yang bisa diunduh kembali kapan saja.

Secara garis besar, sistem ini memiliki dua entitas utama:

- Kategori — digunakan untuk mengelompokkan dokumen, misalnya "Surat Masuk", "Surat Keluar", "Sertifikat", "Kontrak Kerja Sama", dan sebagainya.

- Dokumen — merupakan data arsip itu sendiri, berisi judul dokumen, nomor arsip, deskripsi, berkas file yang diunggah, tanggal arsip, kategori terkait, dan pengguna yang mengunggah dokumen tersebut.

Sepanjang modul ini, seluruh contoh Model, Migration, Controller, Route, dan View akan langsung mengacu pada dua entitas tersebut, sehingga pada akhir modul pembaca telah memiliki sebuah modul CRUD arsip dokumen yang siap dikembangkan lebih lanjut.

1.4 Rancangan Struktur Database

Sebelum masuk ke implementasi, berikut adalah rancangan struktur tabel yang akan digunakan sepanjang modul ini. Struktur ini terdiri atas dua tabel utama, yaitu *kategoris* dan *dokumens*, dengan relasi one-to-many antar keduanya, serta relasi ke tabel bawaan Laravel yaitu *users*.

a. Tabel *kategoris*

database/migrations/xxxx_xx_xx_create_kategoris_table.php

```
Schema::create('kategoris', function (Blueprint $table) {
    $table->id();
    $table->string('nama_kategori', 100);
    $table->text('deskripsi')->nullable();
    $table->timestamps();
});
```

Kolom	Tipe Data	Keterangan
id	BIGINT UNSIGNED (PK, Auto Increment)	Primary key tabel <i>kategoris</i>
nama_kategori	VARCHAR(100)	Nama kategori dokumen, misalnya "Surat Masuk"
deskripsi	TEXT (nullable)	Penjelasan tambahan mengenai kategori, boleh kosong
created_at / updated_at	TIMESTAMP	Dibuat otomatis oleh Laravel melalui <code>timestamps()</code>

b. Tabel *dokumens*

database/migrations/xxxx_xx_xx_create_dokumens_table.php

```
Schema::create('dokumens', function (Blueprint $table) {
    $table->id();
    $table->foreignId('kategori_id')->constrained('kategoris')->onDelete('restrict');
    $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
    $table->string('nomor_arsip', 50)->unique();
    $table->string('judul_dokumen', 255);
    $table->text('deskripsi')->nullable();
    $table->string('nama_file', 255);
    $table->string('tipe_file', 20);
    $table->integer('ukuran_file');
    $table->date('tanggal_arsip');
    $table->timestamps();
});
```

Kolom	Tipe Data	Keterangan
id	BIGINT UNSIGNED (PK, Auto Increment)	Primary key tabel dokumens
kategori_id	BIGINT UNSIGNED (FK → kategoris.id)	Relasi ke kategori dokumen. onDelete('restrict') artinya kategori tidak bisa dihapus selama masih memiliki dokumen terkait
user_id	BIGINT UNSIGNED (FK → users.id)	Relasi ke pengguna yang mengunggah dokumen. onDelete('cascade') artinya jika user dihapus, seluruh dokumen miliknya ikut terhapus
nomor_arsip	VARCHAR(50), UNIQUE	Nomor arsip unik, dibuat otomatis oleh sistem, contoh: ARS-20260831-0001
judul_dokumen	VARCHAR(255)	Judul atau perihal dokumen
deskripsi	TEXT (nullable)	Catatan atau keterangan tambahan dokumen
nama_file	VARCHAR(255)	Nama file hasil unggahan yang tersimpan di server (disarankan nama unik, bukan nama asli)
tipe_file	VARCHAR(20)	Ekstensi berkas, misalnya pdf, docx, jpg
ukuran_file	INTEGER	Ukuran berkas dalam satuan byte
tanggal_arsip	DATE	Tanggal resmi arsip dokumen (bisa berbeda dari tanggal input)
created_at / updated_at	TIMESTAMP	Dibuat otomatis oleh Laravel

CATATAN PENTING

Perhatikan penggunaan onDelete('restrict') pada kategori_id dan onDelete('cascade') pada user_id. Kombinasi ini sengaja dipilih agar data histori arsip tidak pernah kehilangan kategorinya secara tidak sengaja, namun tetap otomatis rapi ketika sebuah akun pengguna dihapus dari sistem. Perbedaan perilaku ini akan kita tangani secara eksplisit pada Bab 8 saat membahas proses hapus data.

1.5 Teknologi yang Digunakan

Komponen	Teknologi	Keterangan
Bahasa Pemrograman	PHP >= 8.2	Wajib untuk menjalankan Laravel 13
Framework	Laravel 13	Framework utama yang digunakan
Database	MySQL / MariaDB	Disarankan menggunakan XAMPP atau Laragon
Front-end	Bootstrap 5 (CDN)	Untuk mempercepat tampilan antarmuka tanpa build tool tambahan
Dependency Manager	Composer	Mengelola package PHP milik Laravel

Komponen	Teknologi	Keterangan
Text Editor	Visual Studio Code	Disarankan, namun bebas menggunakan editor lain

BAB 2 — INSTALASI DAN KONFIGURASI PROJECT

2.1 Persiapan Environment

Sebelum melakukan instalasi Laravel 13, pastikan perangkat sudah memiliki komponen-komponen berikut:

- PHP versi 8.2 atau lebih baru
- Composer (dependency manager PHP)
- Web server lokal seperti XAMPP atau Laragon (untuk MySQL dan Apache)
- Node.js dan NPM (opsional, hanya diperlukan jika ingin melakukan compile asset front-end)

Untuk memastikan PHP dan Composer sudah terpasang dengan benar, jalankan dua perintah berikut pada terminal/CMD:

```
php -v  
composer -v
```

Jika kedua perintah tersebut menampilkan informasi versi tanpa error, maka environment sudah siap digunakan untuk instalasi Laravel 13.

2.2 Instalasi Laravel 13

Ada dua cara umum untuk membuat project Laravel baru, yaitu menggunakan Laravel Installer atau langsung melalui Composer. Modul ini menggunakan cara kedua karena lebih universal di seluruh sistem operasi.

```
composer create-project laravel/laravel siarsip
```

Perintah di atas akan membuat folder baru bernama siarsip yang berisi seluruh struktur dasar project Laravel 13, sekaligus mengunduh seluruh dependency yang dibutuhkan. Nama siarsip dipilih agar konsisten dengan studi kasus Sistem Informasi Arsip Dokumen yang digunakan pada modul ini, namun nama folder tersebut bisa disesuaikan sesuai kebutuhan.

2.3 Menjalankan Project

Setelah proses instalasi selesai, masuk ke dalam folder project, kemudian jalankan built-in server milik Laravel menggunakan Artisan:

```
cd siarsip  
php artisan serve
```

Jika berhasil, Laravel akan menjalankan server lokal pada alamat <http://127.0.0.1:8000>. Buka alamat tersebut pada browser, dan halaman default Laravel akan tampil sebagai tanda instalasi berhasil.

2.4 Konfigurasi Koneksi Database

Selanjutnya kita akan menghubungkan project Laravel 13 dengan database MySQL bernama db_siarsip. Terlebih dahulu, buat database tersebut melalui phpMyAdmin atau perintah SQL. Kemudian buka file .env pada root project, dan cari bagian konfigurasi berikut:

.env (sebelum diubah)

```
DB_CONNECTION=sqlite
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=laravel
# DB_USERNAME=root
# DB_PASSWORD=
```

Ubah konfigurasi tersebut menjadi seperti berikut, disesuaikan dengan studi kasus arsip dokumen:

.env (setelah diubah)

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=db_siarsip
DB_USERNAME=root
DB_PASSWORD=
```

Setelah disimpan, bersihkan cache konfigurasi agar perubahan langsung diterapkan:

```
php artisan config:clear
```

Baris Konfigurasi	Penjelasan
DB_CONNECTION=mysql	Mengubah driver database dari SQLite menjadi MySQL
DB_HOST=127.0.0.1	Alamat server database, localhost pada komputer sendiri
DB_DATABASE=db_siarsip	Nama database yang akan digunakan untuk menyimpan seluruh tabel arsip
DB_USERNAME DB_PASSWORD	/ Kredensial database, default root tanpa password bila menggunakan XAMPP

2.5 Menyiapkan Tampilan dengan Bootstrap 5

Agar modul ini dapat langsung difokuskan pada logika backend tanpa perlu proses build asset yang rumit, tampilan antarmuka menggunakan Bootstrap 5 melalui CDN. Bootstrap akan dipasang langsung di dalam layout utama pada Bab 4, sehingga tidak diperlukan instalasi tambahan melalui NPM.

2.6 Membuat Symbolic Link untuk Penyimpanan File

Karena Sistem Arsip Dokumen akan menyimpan berkas hasil unggahan (PDF, Word, gambar, dan sebagainya), kita perlu menghubungkan folder `storage/app/public` dengan folder `public/storage` agar berkas dapat diakses melalui browser. Jalankan perintah Artisan berikut satu kali saja setelah instalasi:

```
php artisan storage:link
```

CATATAN PENTING

Tanpa menjalankan perintah `storage:link`, berkas dokumen yang berhasil diunggah tidak akan bisa diakses atau diunduh melalui browser, karena secara default folder `storage/app/public` tidak bisa diakses publik. Pastikan langkah ini tidak terlewat sebelum melanjutkan ke Bab 5 (Insert Data).

BAB 3 — DESAIN DATABASE: MODEL DAN MIGRATION

3.1 Konsep Model dan Migration

Model di Laravel digunakan untuk merepresentasikan satu tabel database melalui Eloquent ORM, sehingga operasi CRUD dapat dilakukan dengan sintaks PHP yang rapi tanpa harus menuliskan query SQL secara manual. Sementara itu, Migration digunakan untuk mendefinisikan dan mengelola struktur tabel database melalui kode PHP, sehingga perubahan struktur database dapat dilacak, dibagikan ke seluruh anggota tim, dan dijalankan ulang secara konsisten di environment mana pun.

Pada studi kasus Sistem Arsip Dokumen, kita akan membuat dua pasang Model dan Migration, yaitu Kategori (untuk tabel kategoris) dan Dokumen (untuk tabel dokuments).

3.2 Membuat Model dan Migration Kategori

Jalankan perintah Artisan berikut untuk membuat Model Kategori sekaligus file Migration-nya secara otomatis:

```
php artisan make:model Kategori -m
```

Opsi `-m` (singkatan dari `--migration`) memerintahkan Artisan untuk langsung membuatkan file migration terkait. Perintah di atas akan menghasilkan dua berkas:

- `app/Models/Kategori.php` — berkas Model
- `database/migrations/xxxx_xx_xx_create_kategoris_table.php` — berkas Migration

Laravel secara otomatis menerjemahkan nama Model Kategori menjadi nama tabel kategoris (bentuk jamak/plural dan huruf kecil), sehingga kita tidak perlu mendefinisikan nama tabel secara manual.

3.3 Mengatur Struktur Tabel kategoris

Buka file migration yang baru dibuat, lalu ubah method `up()` menjadi seperti berikut:

database/migrations/xxxx_xx_xx_create_kategoris_table.php

```
public function up(): void
{
    Schema::create('kategoris', function (Blueprint $table) {
        $table->id();
        $table->string('nama_kategori', 100);
        $table->text('deskripsi')->nullable();
        $table->timestamps();
    });
}
```

Kode di atas telah dijelaskan secara rinci pada Tabel 1.4 di Bab 1. Secara ringkas, tabel ini hanya memiliki dua kolom data utama yaitu `nama_kategori` dan `deskripsi`, ditambah kolom bawaan `id` dan `timestamps`.

3.4 Membuat Model dan Migration Dokumen

Selanjutnya, buat Model dan Migration untuk data dokumen arsip:

```
php artisan make:model Dokumen -m
```

Buka file migration create_dokuments_table.php yang baru dibuat, kemudian ubah method up() menjadi:

database/migrations/xxxx_xx_xx_create_dokuments_table.php

```
public function up(): void
{
    Schema::create('dokuments', function (Blueprint $table) {
        $table->id();
        $table->foreignId('kategori_id')->constrained('kategoris')->onDelete('restrict');
        $table->foreignId('user_id')->constrained('users')->onDelete('cascade');
        $table->string('nomor_arsip', 50)->unique();
        $table->string('judul_dokumen', 255);
        $table->text('deskripsi')->nullable();
        $table->string('nama_file', 255);
        $table->string('tipe_file', 20);
        $table->integer('ukuran_file');
        $table->date('tanggal_arsip');
        $table->timestamps();
    });
}
```

Beberapa hal penting yang perlu diperhatikan dari kode di atas:

- `foreignId('kategori_id')->constrained('kategoris')` secara otomatis membuat kolom `kategori_id` bertipe BIGINT UNSIGNED sekaligus mendefinisikan foreign key ke kolom id pada tabel kategoris.
- `onDelete('restrict')` mencegah sebuah baris kategori dihapus apabila masih terdapat dokumen yang menggunakan kategori tersebut — ini penting agar histori arsip tidak pernah kehilangan kategorinya.
- `foreignId('user_id')->constrained('users')` menghubungkan dokumen dengan tabel users bawaan Laravel, yaitu siapa pengguna yang mengunggah/mengarsipkan dokumen tersebut.
- `onDelete('cascade')` pada `user_id` membuat seluruh dokumen milik seorang user otomatis ikut terhapus apabila akun user tersebut dihapus dari sistem.
- `nomor_arsip` diberi properti `->unique()` agar tidak ada dua dokumen dengan nomor arsip yang sama persis.

CATATAN PENTING

Karena tabel dokuments memiliki foreign key ke kategoris dan users, urutan pembuatan tabel menjadi penting. Pastikan file migration kategoris memiliki timestamp (nama file) yang lebih awal dibandingkan file migration dokuments, agar tabel kategoris terbentuk lebih dulu sebelum tabel dokuments mencoba mereferensikannya. Secara default Artisan sudah mengurutkan berdasarkan waktu pembuatan file, sehingga selama Kategori dibuat sebelum Dokumen, urutan ini otomatis benar.

3.5 Menjalankan Migration

Setelah kedua struktur tabel selesai didefinisikan, jalankan perintah berikut untuk benar-benar membuat tabelnya di dalam database db_siarsip:

```
php artisan migrate
```

Jika proses berhasil, Laravel akan menampilkan daftar migration yang sudah dijalankan, dan tabel kategori beserta dokumens akan langsung tampil di phpMyAdmin lengkap dengan relasi foreign key-nya.

3.6 Mendefinisikan Relasi dan Mass Assignment pada Model

Agar Model dapat digunakan untuk operasi CRUD dan relasi antar tabel, kita perlu mendefinisikan properti \$fillable (untuk mass assignment) serta method relasi Eloquent pada masing-masing Model.

a. Model Kategori

app/Models/Kategori.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Kategori extends Model
{
    /**
     * Kolom yang boleh diisi secara mass assignment.
     *
     * @var array
     */
    protected $fillable = [
        'nama_kategori',
        'deskripsi',
    ];

    /**
     * Relasi: satu kategori memiliki banyak dokumen.
     */
    public function dokumens(): HasMany
    {
        return $this->hasMany(Dokumen::class);
    }
}
```

b. Model Dokumen

app/Models/Dokumen.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;
```

```

class Dokumen extends Model
{
    /**
     * Kolom yang boleh diisi secara mass assignment.
     *
     * @var array
     */
    protected $fillable = [
        'kategori_id',
        'user_id',
        'nomor_arsip',
        'judul_dokumen',
        'deskripsi',
        'nama_file',
        'tipe_file',
        'ukuran_file',
        'tanggal_arsip',
    ];

    /**
     * Casting tipe data otomatis.
     *
     * @var array
     */
    protected $casts = [
        'tanggal_arsip' => 'date',
    ];

    /**
     * Relasi: dokumen dimiliki oleh satu kategori.
     */
    public function kategori(): BelongsTo
    {
        return $this->belongsTo(Kategori::class);
    }

    /**
     * Relasi: dokumen diunggah oleh satu user.
     */
    public function user(): BelongsTo
    {
        return $this->belongsTo(User::class);
    }
}

```

Bagian Kode	Penjelasan
\$fillable	Menentukan kolom yang boleh diisi melalui mass assignment, misalnya Kategori::create(\$data) atau Dokumen::create(\$data). Kolom id, created_at, dan updated_at tidak perlu dimasukkan karena diisi otomatis oleh Laravel.
\$casts	Memastikan kolom tanggal_arsip otomatis dikonversi menjadi objek Carbon setiap kali diambil dari database, sehingga dapat langsung diformat pada view, misalnya \$dokumen->tanggal_arsip->format('d-m-Y').
hasMany(Dokumen::class)	Mendefinisikan relasi satu-ke-banyak dari sisi Kategori — satu kategori bisa memiliki banyak dokumen. Digunakan misalnya untuk menampilkan seluruh dokumen dalam satu kategori: \$kategori->dokumens.
belongsTo(Kategori::class)	Mendefinisikan relasi kebalikannya dari sisi Dokumen — satu dokumen dimiliki oleh satu kategori. Digunakan untuk mengambil nama kategori dari sebuah dokumen: \$dokumen->kategori->nama_kategori.

Bagian Kode	Penjelasan
<code>belongsTo(User::class)</code>	Mendefinisikan relasi ke Model User bawaan Laravel, digunakan untuk mengetahui siapa yang mengarsipkan dokumen tersebut: <code>\$dokumen->user->name</code> .

BAB 4 — MENAMPILKAN DATA DARI DATABASE

4.1 Alur MVC yang Digunakan

Untuk menampilkan data, alur yang dilalui pada Laravel adalah Route menerima permintaan dari browser, kemudian meneruskannya ke Controller, Controller mengambil data melalui Model/Eloquent, dan terakhir data tersebut dikirim ke View untuk ditampilkan sebagai HTML. Pola inilah yang akan kita terapkan pada fitur menampilkan daftar kategori dan daftar dokumen arsip.

4.2 Membuat Controller

Buat dua Controller menggunakan Artisan, masing-masing untuk mengelola Kategori dan Dokumen:

```
php artisan make:controller KategoriController --resource
php artisan make:controller DokumenController --resource
```

Opsi `--resource` membuat Controller yang sudah berisi kerangka tujuh method standar CRUD (index, create, store, show, edit, update, destroy), sehingga kita tinggal mengisi logikanya.

4.3 Mendaftarkan Route

Buka file `routes/web.php`, kemudian daftarkan resource route untuk Kategori dan Dokumen, ditambah satu route khusus untuk proses unduh berkas:

routes/web.php

```
<?php

use Illuminate\Support\Facades\Route;
use App\Http\Controllers\KategoriController;
use App\Http\Controllers\DokumenController;

Route::get('/', function () {
    return redirect()->route('dokumen.index');
});

Route::middleware(['auth'])->group(function () {
    Route::resource('kategori', KategoriController::class);
    Route::resource('dokumen', DokumenController::class);
    Route::get('/dokumen/{dokumen}/download', [DokumenController::class, 'download'])
        ->name('dokumen.download');
});

require __DIR__.'/auth.php';
```

Baris Kode	Penjelasan
<code>Route::resource('kategori', ...)</code>	Secara otomatis mendaftarkan tujuh route CRUD sekaligus (index, create, store, show, edit, update, destroy) untuk resource kategori, sesuai konvensi RESTful Laravel.

Baris Kode	Penjelasan
Route::middleware(['auth'])->group(...)	Membungkus seluruh route arsip dengan middleware auth, sehingga hanya pengguna yang sudah login dapat mengakses modul arsip dokumen — sangat penting karena setiap dokumen tercatat memiliki user_id pengunggah.
Route::get('/dokumen/{dokumen}/download', ...)	Route tambahan di luar resource standar, khusus untuk menangani proses unduh berkas dokumen, akan dibahas lebih lanjut pada Bab 6.

CATATAN PENTING

Modul ini mengasumsikan sistem autentikasi (login/register) sudah tersedia, misalnya melalui Laravel Breeze (php artisan install:auth) sehingga tabel users dan proses login sudah berjalan. Pembahasan instalasi starter kit autentikasi berada di luar cakupan modul ini karena berfokus pada konsep Model, Migration, dan CRUD.

4.4 Membuat Layout Utama (Master Layout)

Agar tampilan konsisten di seluruh halaman, kita buat satu layout utama menggunakan Blade Component/Layout yang memuat navbar, sidebar sederhana, serta area flash message.

resources/views/layouts/app.blade.php

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <title>SIARSIP - @yield('title', 'Sistem Arsip Dokumen')</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet">
</head>
<body class="bg-light">
  <nav class="navbar navbar-dark bg-dark px-3">
    <span class="navbar-brand mb-0 h1">SIARSIP</span>
    <div>
      <a href="{{ route('dokumen.index') }}" class="btn btn-sm btn-outline-light me-2">Dokumen</a>
      <a href="{{ route('kategori.index') }}" class="btn btn-sm btn-outline-light">Kategori</a>
    </div>
  </nav>

  <div class="container py-4">
    @if (session('success'))
      <div class="alert alert-success">{{ session('success') }}</div>
    @endif

    @if (session('error'))
      <div class="alert alert-danger">{{ session('error') }}</div>
    @endif

    @yield('content')
  </div>
</body>
</html>
```


4.5 Menampilkan Daftar Kategori

Isi method `index()` pada `KategoriController` untuk mengambil seluruh data kategori beserta jumlah dokumen pada masing-masing kategori:

app/Http/Controllers/KategoriController.php

```
public function index()
{
    $kategoris = Kategori::withCount('dokumens')->latest()->paginate(10);

    return view('kategori.index', compact('kategoris'));
}
```

`withCount('dokumens')` memanfaatkan relasi `hasMany` yang telah didefinisikan pada Bab 3 untuk menghitung jumlah dokumen di setiap kategori tanpa perlu query tambahan secara manual, hasilnya bisa diakses melalui `$kategoris->dokumens_count`.

resources/views/kategori/index.blade.php

```
@extends('layouts.app')

@section('content')
<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Data Kategori Arsip</h4>
    <a href="{{ route('kategori.create') }}" class="btn btn-primary">+ Tambah Kategori</a>
</div>

<table class="table table-bordered bg-white">
    <thead>
        <tr>
            <th>No</th>
            <th>Nama Kategori</th>
            <th>Deskripsi</th>
            <th>Jumlah Dokumen</th>
            <th width="160">Aksi</th>
        </tr>
    </thead>
    <tbody>
        @forelse ($kategoris as $kategori)
            <tr>
                <td>{{ $loop->iteration }}</td>
                <td>{{ $kategori->nama_kategori }}</td>
                <td>{{ $kategori->deskripsi ?? '-' }}</td>
                <td>{{ $kategori->dokumens_count }}</td>
                <td>
                    <a href="{{ route('kategori.edit', $kategori->id) }}" class="btn btn-sm btn-warning">Edit</a>
                    <form action="{{ route('kategori.destroy', $kategori->id) }}" method="POST"
                    class="d-inline"
                        onsubmit="return confirm('Yakin ingin menghapus kategori ini?')">
                        @csrf
                        @method('DELETE')
                        <button type="submit" class="btn btn-sm btn-danger">Hapus</button>
                    </form>
                </td>
            </tr>
        @empty
            <tr><td colspan="5" class="text-center">Belum ada data kategori.</td></tr>
        @endforelse
    </tbody>
</table>
```

```
{{ $kategoris->links() }}
@endsection
```

4.6 Menampilkan Daftar Dokumen Arsip

Untuk halaman utama arsip dokumen, kita tambahkan fitur pencarian judul dan filter berdasarkan kategori, sekaligus melakukan eager loading relasi agar query lebih efisien (menghindari masalah N+1 query).

app/Http/Controllers/DokumenController.php

```
public function index(Request $request)
{
    $dokumens = Dokumen::with(['kategori', 'user'])
        ->when($request->cari, function ($query, $cari) {
            $query->where('judul_dokumen', 'like', "%{$cari}%")
                ->orWhere('nomor_arsip', 'like', "%{$cari}%");
        })
        ->when($request->kategori_id, function ($query, $kategoriId) {
            $query->where('kategori_id', $kategoriId);
        })
        ->latest('tanggal_arsip')
        ->paginate(10)
        ->withQueryString();

    $kategoris = Kategori::orderBy('nama_kategori')->get();

    return view('dokumen.index', compact('dokumens', 'kategoris'));
}
```

Bagian Kode	Penjelasan
<code>with(['kategori', 'user'])</code>	Eager loading — mengambil data kategori dan user terkait dalam satu proses query tambahan yang efisien, alih-alih melakukan query berulang setiap kali <code>\$dokumen->kategori</code> dipanggil di dalam perulangan view.
<code>when(\$request->cari, ...)</code>	Menambahkan kondisi where pencarian hanya apabila parameter cari dikirimkan melalui form/URL, sehingga controller tetap ringkas tanpa if-else manual.
<code>withQueryString()</code>	Memastikan parameter pencarian dan filter tetap terbawa saat berpindah halaman pagination.

resources/views/dokumen/index.blade.php

```
@extends('layouts.app')

@section('content')
<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Data Arsip Dokumen</h4>
    <a href="{{ route('dokumen.create') }}" class="btn btn-primary">+ Tambah Arsip</a>
</div>

<form method="GET" class="row g-2 mb-3">
    <div class="col-md-5">
        <input type="text" name="cari" value="{{ request('cari') }}" class="form-control"
            placeholder="Cari judul atau nomor arsip...">
    </div>
</form>
```

```

</div>
<div class="col-md-4">
  <select name="kategori_id" class="form-select">
    <option value="">Semua Kategori</option>
    @foreach ($kategoris as $kategori)
      <option value="{{ $kategori->id }}" @selected(request('kategori_id') ==
$kategori->id)>
        {{ $kategori->nama_kategori }}
      </option>
    @endforeach
  </select>
</div>
<div class="col-md-3">
  <button type="submit" class="btn btn-secondary w-100">Filter</button>
</div>
</form>

<table class="table table-bordered bg-white align-middle">
  <thead>
    <tr>
      <th>Nomor Arsip</th>
      <th>Judul Dokumen</th>
      <th>Kategori</th>
      <th>Tanggal Arsip</th>
      <th>Diunggah Oleh</th>
      <th width="180">Aksi</th>
    </tr>
  </thead>
  <tbody>
    @forelse ($dokumens as $dokumen)
      <tr>
        <td><span class="badge bg-secondary">{{ $dokumen->nomor_arsip }}</span></td>
        <td>{{ $dokumen->judul_dokumen }}</td>
        <td>{{ $dokumen->kategori->nama_kategori }}</td>
        <td>{{ $dokumen->tanggal_arsip->format('d-m-Y') }}</td>
        <td>{{ $dokumen->user->name }}</td>
        <td>
          <a href="{{ route('dokumen.show', $dokumen->id) }}" class="btn btn-sm btn-
info">Detail</a>
          <a href="{{ route('dokumen.edit', $dokumen->id) }}" class="btn btn-sm btn-
warning">Edit</a>
        </td>
      </tr>
    @empty
      <tr><td colspan="6" class="text-center">Belum ada data arsip dokumen.</td></tr>
    @endforelse
  </tbody>
</table>

{{ $dokumens->links() }}
@endsection

```

BAB 5 — INSERT DATA (TAMBAH ARSIP DOKUMEN)

5.1 Tambah Data Kategori

Sebelum menambahkan dokumen, kategori dokumen perlu tersedia terlebih dahulu karena kolom `kategori_id` bersifat wajib (foreign key). Lengkapi method `create()` dan `store()` pada `KategoriController`:

app/Http/Controllers/KategoriController.php

```
public function create()
{
    return view('kategori.create');
}

public function store(Request $request)
{
    $request->validate([
        'nama_kategori' => 'required|string|max:100|unique:kategoris,nama_kategori',
        'deskripsi'     => 'nullable|string',
    ]);

    Kategori::create($request->only(['nama_kategori', 'deskripsi']));

    return redirect()->route('kategori.index')
        ->with('success', 'Kategori berhasil ditambahkan.');
```

resources/views/kategori/create.blade.php

```
@extends('layouts.app')

@section('content')
<h4 class="mb-3">Tambah Kategori</h4>

<form action="{{ route('kategori.store') }}" method="POST" class="bg-white p-4 rounded
shadow-sm">
    @csrf
    <div class="mb-3">
        <label class="form-label">Nama Kategori</label>
        <input type="text" name="nama_kategori" value="{{ old('nama_kategori') }}"
            class="form-control @error('nama_kategori') is-invalid @enderror">
        @error('nama_kategori') <div class="invalid-feedback">{{ $message }}</div> @enderror
    </div>
    <div class="mb-3">
        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi')
    }}</textarea>
    </div>
    <button type="submit" class="btn btn-primary">Simpan</button>
    <a href="{{ route('kategori.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection
```

5.2 Tambah Data Dokumen Arsip (dengan Upload File)

Ini adalah bagian inti dari studi kasus Sistem Arsip Dokumen, karena setiap dokumen wajib disertai berkas fisik yang diunggah. Ada tiga hal khusus yang perlu ditangani pada proses ini:

- Nomor arsip (nomor_arsip) dibuat otomatis oleh sistem, bukan diinput manual oleh pengguna, sehingga formatnya konsisten dan tidak mudah tertukar.
- Berkas yang diunggah disimpan dengan nama unik menggunakan Str::uuid(), sedangkan judul dokumen (judul_dokumen) tetap menyimpan nama/perihal yang mudah dibaca manusia. Ini mencegah dua file dengan nama asli yang sama saling menimpa.
- Kolom tipe_file dan ukuran_file diisi otomatis berdasarkan metadata file yang diunggah, tidak diinput manual.

Lengkapi method create() pada DokumenController untuk menampilkan form, lengkap dengan daftar kategori sebagai pilihan dropdown:

app/Http/Controllers/DokumenController.php

```
public function create()
{
    $kategoris = Kategori::orderBy('nama_kategori')->get();

    return view('dokumen.create', compact('kategoris'));
}
```

Selanjutnya lengkapi method store() dengan logika validasi, pembuatan nomor arsip otomatis, dan penyimpanan berkas ke disk public:

app/Http/Controllers/DokumenController.php

```
use Illuminate\Support\Str;
use Illuminate\Support\Facades\Storage;

public function store(Request $request)
{
    $request->validate([
        'kategori_id' => 'required|exists:kategoris,id',
        'judul_dokumen' => 'required|string|max:255',
        'deskripsi' => 'nullable|string',
        'tanggal_arsip' => 'required|date',
        'file' =>
        'required|file|mimes:pdf,doc,docx,xls,xlsx,jpg,jpeg,png|max:10240',
    ]);

    // 1. Buat nomor arsip otomatis, format: ARS-YYYYMMDD-0001
    $tanggal = now()->format('Ymd');
    $urutan = Dokumen::whereDate('created_at', now())->count() + 1;
    $nomorArsip = 'ARS-' . $tanggal . '-' . str_pad($urutan, 4, '0', STR_PAD_LEFT);

    // 2. Simpan berkas dengan nama unik ke storage/app/public/dokumen
    $file = $request->file('file');
    $ekstensi = $file->getClientOriginalExtension();
    $namaFileTersimpan = Str::uuid() . '.' . $ekstensi;
    $file->storeAs('dokumen', $namaFileTersimpan, 'public');

    // 3. Simpan data ke database
    Dokumen::create([
        'kategori_id' => $request->kategori_id,
        'user_id' => auth()->id(),
        'nomor_arsip' => $nomorArsip,
        'judul_dokumen' => $request->judul_dokumen,
        'deskripsi' => $request->deskripsi,
        'nama_file' => $namaFileTersimpan,
```

```

        'tipe_file'      => $ekstensi,
        'ukuran_file'   => $file->getSize(),
        'tanggal_arsip' => $request->tanggal_arsip,
    ]);

    return redirect()->route('dokumen.index')
        ->with('success', 'Dokumen berhasil diarsipkan dengan nomor ' . $nomorArsip);
}

```

Bagian Kode	Penjelasan
mimes:pdf,doc,docx,xls,xlsx,jpg,jpeg,png max:10240	Membatasi jenis berkas yang boleh diunggah hanya format dokumen dan gambar umum, dengan ukuran maksimal 10240 KB (10 MB), untuk mencegah penyalahgunaan upload.
Dokumen::whereDate('created_at', now()->count()+ 1	Menghitung jumlah dokumen yang sudah diarsipkan pada hari yang sama, lalu menambah 1 sebagai nomor urut berikutnya, sehingga nomor arsip selalu unik per hari.
str_pad(\$urutan, 4, '0', STR_PAD_LEFT)	Mengubah angka urutan menjadi 4 digit dengan tambahan angka nol di depan, misalnya 1 menjadi 0001, agar format nomor arsip rapi dan konsisten.
Str::uuid() . '.' . \$ekstensi	Membuat nama file unik menggunakan UUID (Universally Unique Identifier) sehingga dua berkas dengan nama asli sama tidak akan saling menimpa satu sama lain di server.
\$file->storeAs('dokumen', \$namaFileTersimpan, 'public')	Menyimpan berkas ke folder storage/app/public/dokumen, yang dapat diakses publik melalui folder public/storage/dokumen berkat symbolic link yang sudah dibuat pada Bab 2.
auth()->id()	Mengambil ID pengguna yang sedang login untuk disimpan sebagai user_id, sesuai dengan struktur tabel dokumens yang mewajibkan relasi ke pengguna pengunggah.

resources/views/dokumen/create.blade.php

```

@extends('layouts.app')

@section('content')
<h4 class="mb-3">Tambah Arsip Dokumen</h4>

<form action="{{ route('dokumen.store') }}" method="POST" enctype="multipart/form-data"
    class="bg-white p-4 rounded shadow-sm">
    @csrf

    <div class="mb-3">
        <label class="form-label">Kategori</label>
        <select name="kategori_id" class="form-select @error('kategori_id') is-invalid
@enderror">
            <option value="">-- Pilih Kategori --</option>
            @foreach ($kategoris as $kategori)
            <option value="{{ $kategori->id }}" @selected(old('kategori_id') == $kategori-
>id)>
                {{ $kategori->nama_kategori }}
            </option>
            @endforeach

```

```

        </select>
        @error('kategori_id') <div class="invalid-feedback">{{ $message }}</div> @enderror
    </div>

    <div class="mb-3">
        <label class="form-label">Judul Dokumen</label>
        <input type="text" name="judul_dokumen" value="{{ old('judul_dokumen') }}"
            class="form-control @error('judul_dokumen') is-invalid @enderror">
        @error('judul_dokumen') <div class="invalid-feedback">{{ $message }}</div> @enderror
    </div>

    <div class="mb-3">
        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi')
    }}</textarea>
    </div>

    <div class="mb-3">
        <label class="form-label">Tanggal Arsip</label>
        <input type="date" name="tanggal_arsip" value="{{ old('tanggal_arsip') }}"
            class="form-control @error('tanggal_arsip') is-invalid @enderror">
        @error('tanggal_arsip') <div class="invalid-feedback">{{ $message }}</div> @enderror
    </div>

    <div class="mb-3">
        <label class="form-label">Berkas Dokumen (PDF/Word/Excel/Gambar, maks 10MB)</label>
        <input type="file" name="file" class="form-control @error('file') is-invalid
    @enderror">
        @error('file') <div class="invalid-feedback">{{ $message }}</div> @enderror
    </div>

    <button type="submit" class="btn btn-primary">Simpan Arsip</button>
    <a href="{{ route('dokumen.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection

```

CATATAN PENTING

Perhatikan atribut `enctype="multipart/form-data"` pada tag `<form>`. Atribut ini wajib ditambahkan setiap kali form memiliki input bertipe file, karena tanpa atribut ini browser tidak akan mengirimkan data berkas ke server, dan `$request->file('file')` akan selalu bernilai null.

BAB 6 — MENAMPILKAN DETAIL DATA DAN UNDUH BERKAS

6.1 Menampilkan Detail Dokumen

Setiap baris data arsip perlu dapat dilihat detailnya secara lengkap, termasuk metadata berkas (tipe file dan ukuran file) serta tombol untuk mengunduh berkas aslinya. Lengkapi method `show()` pada `DokumenController`:

app/Http/Controllers/DokumenController.php

```
public function show(string $id)
{
    $dokumen = Dokumen::with(['kategori', 'user'])->findOrFail($id);

    return view('dokumen.show', compact('dokumen'));
}
```

`findOrFail($id)` akan otomatis menampilkan halaman error 404 apabila data dengan id tersebut tidak ditemukan, sehingga kita tidak perlu menuliskan pengecekan manual.

resources/views/dokumen/show.blade.php

```
@extends('layouts.app')

@section('content')
<div class="d-flex justify-content-between align-items-center mb-3">
    <h4 class="mb-0">Detail Arsip Dokumen</h4>
    <a href="{{ route('dokumen.index') }}" class="btn btn-secondary btn-sm">&larr;
    Kembali</a>
</div>

<div class="card">
    <div class="card-body">
        <table class="table table-borderless mb-0">
            <tr>
                <th width="220">Nomor Arsip</th>
                <td><span class="badge bg-secondary">{{ $dokumen->nomor_arsip }}</span></td>
            </tr>
            <tr>
                <th>Judul Dokumen</th>
                <td>{{ $dokumen->judul_dokumen }}</td>
            </tr>
            <tr>
                <th>Kategori</th>
                <td>{{ $dokumen->kategori->nama_kategori }}</td>
            </tr>
            <tr>
                <th>Deskripsi</th>
                <td>{{ $dokumen->deskripsi ?? '-' }}</td>
            </tr>
            <tr>
                <th>Tanggal Arsip</th>
                <td>{{ $dokumen->tanggal_arsip->format('d F Y') }}</td>
            </tr>
            <tr>
                <th>Diunggah Oleh</th>
                <td>{{ $dokumen->user->name }}</td>
            </tr>
        </table>
    </div>
</div>
```



```

        <th>Tipe Berkas</th>
        <td>: {{ strtoupper($dokumen->tipe_file) }}</td>
    </tr>
    <tr>
        <th>Ukuran Berkas</th>
        <td>: {{ number_format($dokumen->ukuran_file / 1024, 2) }} KB</td>
    </tr>
</table>

@if (in_array($dokumen->tipe_file, ['jpg', 'jpeg', 'png']))
<div class="my-3">
    
    </div>
@endif

    <a href="{{ route('dokumen.download', $dokumen->id) }}" class="btn btn-primary">
        Unduh Berkas
    </a>
</div>
</div>
@endsection

```

CATATAN PENTING

Kode {{ asset('storage/dokumen/' . \$dokumen->nama_file) }} hanya akan berfungsi apabila perintah php artisan storage:link pada Bab 2 telah dijalankan, karena folder public/storage merupakan symbolic link menuju storage/app/public.

6.2 Mengunduh Berkas Dokumen

Alih-alih mengakses file secara langsung melalui URL publik (yang membuat nama file UUID acak terekspos begitu saja), kita buat satu route dan method khusus untuk proses unduh, sehingga nama berkas yang diterima pengguna tetap terlihat rapi sesuai judul dokumen aslinya:

app/Http/Controllers/DokumenController.php

```

public function download(Dokumen $dokumen)
{
    $path = 'dokumen/' . $dokumen->nama_file;

    if (! Storage::disk('public')->exists($path)) {
        abort(404, 'Berkas tidak ditemukan di server.');
```

Bagian Kode	Penjelasan
<pre>public function download(Dokumen \$dokumen)</pre>	Menggunakan Route Model Binding — Laravel otomatis mengambil data Dokumen berdasarkan id pada URL tanpa perlu findOrFail() secara manual.
<pre>Storage::disk('public')->exists(\$path)</pre>	Memastikan berkas benar-benar masih ada secara fisik sebelum diunduh, mengantisipasi kondisi berkas terhapus manual dari server namun datanya masih tercatat di database.
<pre>Str::slug(\$dokumen->judul_dokumen)</pre>	Mengubah judul dokumen menjadi format nama file yang aman (tanpa spasi/symbol khusus), sehingga saat diunduh pengguna melihat nama file yang deskriptif, bukan kode UUID acak.
<pre>Storage::disk('public')->download(...)</pre>	Method bawaan Laravel untuk mengirimkan file sebagai response unduhan (force download) lengkap dengan header Content-Disposition yang sesuai.

BAB 7 — EDIT DAN UPDATE DATA

7.1 Edit dan Update Data Kategori

Proses edit kategori relatif sederhana karena tidak melibatkan berkas. Lengkapi method `edit()` dan `update()` pada `KategoriController`:

app/Http/Controllers/KategoriController.php

```
public function edit(string $id)
{
    $kategori = Kategori::findOrFail($id);

    return view('kategori.edit', compact('kategori'));
}

public function update(Request $request, string $id)
{
    $kategori = Kategori::findOrFail($id);

    $request->validate([
        'nama_kategori' => 'required|string|max:100|unique:kategoris,nama_kategori,' .
        $kategori->id,
        'deskripsi' => 'nullable|string',
    ]);

    $kategori->update($request->only(['nama_kategori', 'deskripsi']));

    return redirect()->route('kategori.index')
        ->with('success', 'Kategori berhasil diperbarui.');
```

CATATAN PENTING

Perhatikan aturan `unique:kategoris,nama_kategori,' . $kategori->id` — parameter ketiga ini memberitahu Laravel untuk mengecualikan baris kategori yang sedang diedit dari pengecekan `unique`, sehingga pengguna tidak akan mendapat error validasi hanya karena tidak mengubah nama kategorinya sama sekali.

resources/views/kategori/edit.blade.php

```
@extends('layouts.app')

@section('content')
<h4 class="mb-3">Edit Kategori</h4>

<form action="{{ route('kategori.update', $kategori->id) }}" method="POST"
    class="bg-white p-4 rounded shadow-sm">
    @csrf
    @method('PUT')
    <div class="mb-3">
        <label class="form-label">Nama Kategori</label>
        <input type="text" name="nama_kategori" value="{{ old('nama_kategori', $kategori-
        >nama_kategori) }}"
            class="form-control @error('nama_kategori') is-invalid @enderror">
        @error('nama_kategori') <div class="invalid-feedback">{{ $message }}</div> @enderror
    </div>
    <div class="mb-3">
```

```

        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi',
$kategori->deskripsi) }}</textarea>
    </div>
    <button type="submit" class="btn btn-primary">Update</button>
    <a href="{{ route('kategori.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection

```

7.2 Edit dan Update Data Dokumen (Berkas Bersifat Opsional)

Pada proses edit dokumen, terdapat aturan khusus: pengguna tidak wajib mengunggah ulang berkas. Apabila tidak ada berkas baru yang dipilih, sistem harus mempertahankan berkas lama. Namun apabila pengguna mengunggah berkas baru, sistem wajib menghapus berkas lama dari storage agar tidak menumpuk file yang sudah tidak terpakai (orphan file).

app/Http/Controllers/DokumenController.php

```

public function edit(string $id)
{
    $dokumen = Dokumen::findOrFail($id);
    $kategoris = Kategori::orderBy('nama_kategori')->get();

    return view('dokumen.edit', compact('dokumen', 'kategoris'));
}

public function update(Request $request, string $id)
{
    $dokumen = Dokumen::findOrFail($id);

    $request->validate([
        'kategori_id' => 'required|exists:kategoris,id',
        'judul_dokumen' => 'required|string|max:255',
        'deskripsi' => 'nullable|string',
        'tanggal_arsip' => 'required|date',
        'file' =>
'nullable|file|mimes:pdf,doc,docx,xls,xlsx,jpg,jpeg,png|max:10240',
    ]);

    $data = $request->only(['kategori_id', 'judul_dokumen', 'deskripsi', 'tanggal_arsip']);

    // Jika pengguna mengunggah berkas baru, ganti berkas lama
    if ($request->hasFile('file')) {
        if (Storage::disk('public')->exists('dokumen/' . $dokumen->nama_file)) {
            Storage::disk('public')->delete('dokumen/' . $dokumen->nama_file);
        }

        $file = $request->file('file');
        $ekstensi = $file->getClientOriginalExtension();
        $namaFileTersimpan = Str::uuid() . '.' . $ekstensi;
        $file->storeAs('dokumen', $namaFileTersimpan, 'public');

        $data['nama_file'] = $namaFileTersimpan;
        $data['tipe_file'] = $ekstensi;
        $data['ukuran_file'] = $file->getSize();
    }

    $dokumen->update($data);
}

```

```

return redirect()->route('dokumen.index')
->with('success', 'Dokumen berhasil diperbarui.');
```

Bagian Kode	Penjelasan
'file' => 'nullable file ...'	Berbeda dengan aturan pada store() yang bersifat required, di sini menggunakan nullable karena file boleh tidak dikirim ulang oleh pengguna.
if (\$request->hasFile('file'))	Blok ini hanya dijalankan apabila pengguna benar-benar memilih berkas baru pada form edit. Jika tidak, variabel \$data tidak memiliki key nama_file/tipe_file/ukuran_file, sehingga kolom tersebut tidak ikut ter-update dan nilai lama tetap dipertahankan.
Storage::disk('public')->delete(...)	Menghapus berkas lama dari server sebelum menyimpan berkas baru, mencegah penumpukan file yang sudah tidak lagi direferensikan oleh data manapun.

resources/views/dokumen/edit.blade.php

```

@extends('layouts.app')

@section('content')
<h4 class="mb-3">Edit Arsip Dokumen</h4>

<form action="{{ route('dokumen.update', $dokumen->id) }}" method="POST"
enctype="multipart/form-data"
class="bg-white p-4 rounded shadow-sm">
    @csrf
    @method('PUT')

    <div class="mb-3">
        <label class="form-label">Kategori</label>
        <select name="kategori_id" class="form-select">
            @foreach ($kategoris as $kategori)
                <option value="{{ $kategori->id }}"
                    @selected(old('kategori_id', $dokumen->kategori_id) == $kategori->id)>
                    {{ $kategori->nama_kategori }}
            </option>
            @endforeach
        </select>
    </div>

    <div class="mb-3">
        <label class="form-label">Judul Dokumen</label>
        <input type="text" name="judul_dokumen"
            value="{{ old('judul_dokumen', $dokumen->judul_dokumen) }}" class="form-
control">
    </div>

    <div class="mb-3">
        <label class="form-label">Deskripsi</label>
        <textarea name="deskripsi" rows="3" class="form-control">{{ old('deskripsi',
$dokumen->deskripsi) }}</textarea>
    </div>

    <div class="mb-3">
        <label class="form-label">Tanggal Arsip</label>
        <input type="date" name="tanggal_arsip"
            value="{{ old('tanggal_arsip', $dokumen->tanggal_arsip->format('Y-m-d')) }}"
class="form-control">
    </div>
</form>

```

```
</div>

<div class="mb-3">
  <label class="form-label">Berkas Saat Ini</label>
  <p class="mb-1">
    <a href="{{ route('dokumen.download', $dokumen->id) }}" target="_blank">
      {{ $dokumen->judul_dokumen }}.{{ $dokumen->tipe_file }}
    </a>
  </p>
  <label class="form-label">Ganti Berkas (kosongkan jika tidak ingin mengubah)</label>
  <input type="file" name="file" class="form-control">
</div>

<button type="submit" class="btn btn-primary">Update</button>
<a href="{{ route('dokumen.index') }}" class="btn btn-secondary">Batal</a>
</form>
@endsection
```

BAB 8 — DELETE DATA (HAPUS ARSIP)

8.1 Menghapus Data Kategori (Menangani Relasi Restrict)

Karena kolom `kategori_id` pada tabel `dokumens` menggunakan `onDelete('restrict')`, database MySQL akan menolak proses penghapusan kategori yang masih memiliki dokumen terkait, dan melemparkan `Illuminate\Database\QueryException`. Kita perlu menangkap exception tersebut agar aplikasi menampilkan pesan yang ramah pengguna, bukan halaman error 500.

app/Http/Controllers/KategoriController.php

```
use Illuminate\Database\QueryException;

public function destroy(string $id)
{
    $kategori = Kategori::findOrFail($id);

    try {
        $kategori->delete();
    } catch (QueryException $e) {
        return redirect()->route('kategori.index')
            ->with('error', 'Kategori "' . $kategori->nama_kategori . '" tidak dapat dihapus
            . 'karena masih memiliki dokumen arsip yang terkait.');
```

CATATAN PENTING

Sebagai alternatif yang lebih eksplisit dan tidak bergantung pada exception database, kategori juga bisa dicek terlebih dahulu menggunakan `if ($kategori->dokumens()->exists())` sebelum proses `delete()` dijalankan. Pendekatan try-catch pada contoh di atas dipilih agar aturan integritas data tetap dijamin langsung oleh database (constraint), sehingga lebih aman meskipun ada jalur kode lain yang mencoba menghapus data secara langsung.

8.2 Menghapus Data Dokumen Beserta Berkas Fisiknya

Berbeda dengan kategori, penghapusan dokumen tidak dibatasi oleh foreign key manapun (karena tidak ada tabel lain yang mereferensikan `dokumens`). Namun kita tetap perlu memastikan berkas fisik di storage ikut terhapus, agar tidak menyisakan file yatim (orphan file) yang memenuhi ruang penyimpanan server.

app/Http/Controllers/DokumenController.php

```
public function destroy(string $id)
{
    $dokumen = Dokumen::findOrFail($id);

    if (Storage::disk('public')->exists('dokumen/' . $dokumen->nama_file)) {
        Storage::disk('public')->delete('dokumen/' . $dokumen->nama_file);
    }
```

```
$dokumen->delete();

return redirect()->route('dokumen.index')
    ->with('success', 'Arsip dokumen berhasil dihapus.');
```

8.3 Menambahkan Konfirmasi Hapus pada View

Untuk mencegah penghapusan data yang tidak disengaja, setiap tombol hapus dibungkus dengan form beserta konfirmasi JavaScript sederhana. Tambahkan potongan berikut pada tabel `resources/views/dokumen/index.blade.php` di kolom Aksi:

resources/views/dokumen/index.blade.php (tambahan kolom aksi)

```
<form action="{{ route('dokumen.destroy', $dokumen->id) }}" method="POST" class="d-inline"
    onsubmit="return confirm('Yakin ingin menghapus arsip \'' + '{{ $dokumen->judul_dokumen
}}' + '\''? Berkas terkait juga akan terhapus permanen.')">
    @csrf
    @method('DELETE')
    <button type="submit" class="btn btn-sm btn-danger">Hapus</button>
</form>
```

`@method('DELETE')` diperlukan karena HTML form secara native hanya mendukung method GET dan POST. Laravel menyediakan Blade directive ini untuk melakukan HTTP method spoofing, sehingga permintaan tetap dikirim sebagai POST namun dikenali oleh Router Laravel sebagai method DELETE sesuai dengan resource route yang telah didaftarkan pada Bab 4.

BAB 9 — PENYEMPURNAAN: VALIDASI, KEAMANAN, DAN AKSES

9.1 Merapikan Validasi dengan Form Request

Setelah aturan validasi pada method `store()` dan `update()` `DokumenController` semakin banyak, praktik yang lebih rapi adalah memindahkannya ke kelas `Form Request` tersendiri, sehingga `Controller` hanya fokus pada logika bisnis. Buat `Form Request` menggunakan `Artisan`:

```
php artisan make:request StoreDokumenRequest
```

app/Http/Requests/StoreDokumenRequest.php

```
<?php

namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StoreDokumenRequest extends FormRequest
{
    public function authorize(): bool
    {
        return auth()->check();
    }

    public function rules(): array
    {
        return [
            'kategori_id' => 'required|exists:kategoris,id',
            'judul_dokumen' => 'required|string|max:255',
            'deskripsi' => 'nullable|string',
            'tanggal_arsip' => 'required|date',
            'file' =>
                'required|file|mimes:pdf,doc,docx,xls,xlsx,jpg,jpeg,png|max:10240',
        ];
    }

    public function messages(): array
    {
        return [
            'file.mimes' => 'Berkas harus berformat PDF, Word, Excel, atau gambar (JPG/PNG).',
            'file.max' => 'Ukuran berkas tidak boleh lebih dari 10 MB.',
        ];
    }
}
```

Setelah kelas ini dibuat, method `store()` pada `Controller` cukup menerima tipe `StoreDokumenRequest` `$request` sebagai pengganti `Request` `$request`, dan pemanggilan `$request->validate(...)` tidak lagi diperlukan karena validasi sudah otomatis dijalankan lebih dulu oleh `Laravel` sebelum method `Controller` dieksekusi.

9.2 Middleware Autentikasi

Seluruh route arsip dokumen wajib dilindungi middleware `auth`, karena kolom `user_id` pada tabel `dokumens` mensyaratkan adanya pengguna yang sedang login. Konfigurasi ini telah diterapkan pada Bab 4 melalui

`Route::middleware(['auth'])->group(...)`. Selain itu, pastikan setiap query data dokumen turut mempertimbangkan siapa yang login, terutama apabila di masa depan modul ini dikembangkan menjadi multi-user dengan hak akses berbeda-beda, misalnya melalui package otorisasi seperti Laravel Policy atau Spatie Laravel-Permission.

9.3 Catatan Keamanan Upload Berkas

- Selalu validasi ekstensi file menggunakan aturan mimes, jangan hanya mengandalkan nama field input pada form karena nama field dapat dimanipulasi.
- Jangan pernah menyimpan berkas menggunakan nama asli dari pengguna secara langsung (`getClientOriginalName()`) sebagai nama file di server — selain berisiko collision antar file, nama asli juga bisa mengandung karakter berbahaya. Gunakan nama unik seperti `Str::uuid()` sebagaimana diterapkan pada Bab 5.
- Tentukan batas ukuran maksimal file (`max:10240` dalam contoh ini setara 10 MB) untuk menghindari penyalahgunaan penyimpanan server.
- Simpan berkas arsip di disk public hanya jika memang dokumen tersebut boleh diakses lewat tautan langsung. Untuk dokumen yang bersifat rahasia/terbatas, sebaiknya gunakan disk local dan sajikan berkas melalui Controller (misalnya method `download()` pada Bab 6) yang memverifikasi hak akses pengguna terlebih dahulu, bukan disk public yang dapat diakses siapa saja yang mengetahui URL-nya.

9.4 Flash Message

Sepanjang modul ini, setiap redirect setelah proses simpan, ubah, dan hapus data selalu disertai `->with('success', ...)` atau `->with('error', ...)`. Pesan tersebut ditangkap dan ditampilkan otomatis melalui blok `@if (session('success'))` dan `@if (session('error'))` yang telah didefinisikan pada layout utama `resources/views/layouts/app.blade.php` di Bab 4, sehingga tidak perlu menuliskan ulang alert di setiap halaman.

BAB 10 — PENUTUP

10.1 Ringkasan

Melalui modul ini, kita telah membangun sebuah modul CRUD Sistem Informasi Arsip Dokumen (SIARSIP) secara lengkap menggunakan Laravel 13, mulai dari:

- Instalasi dan konfigurasi project Laravel 13 beserta koneksi database MySQL.
- Perancangan Model dan Migration untuk dua entitas utama, yaitu Kategori dan Dokumen, lengkap dengan relasi foreign key dan strategi onDelete restrict/cascade yang sesuai kebutuhan bisnis arsip.
- Penerapan relasi Eloquent hasMany dan belongsTo antar Model, termasuk relasi ke Model User bawaan Laravel.
- Menampilkan data dengan fitur pencarian, filter kategori, dan pagination.
- Insert data disertai proses upload berkas, pembuatan nomor arsip otomatis, dan penamaan file yang aman menggunakan UUID.
- Menampilkan detail data beserta fitur unduh dan preview berkas.
- Edit dan update data dengan penanganan khusus untuk berkas yang bersifat opsional.
- Delete data dengan penanganan constraint database (restrict) serta pembersihan berkas fisik dari storage.
- Penyempurnaan melalui Form Request, middleware autentikasi, dan praktik keamanan upload berkas.

Dengan menyelesaikan seluruh bab pada modul ini, pembaca telah memahami alur kerja Laravel secara end-to-end pada kasus nyata yang melibatkan relasi antar tabel dan pengelolaan berkas, bukan sekadar CRUD teks biasa.

10.2 Pengembangan Lanjutan

Sistem Arsip Dokumen yang telah dibangun pada modul ini dapat dikembangkan lebih lanjut, di antaranya:

- Ekspor data arsip ke PDF atau Excel untuk kebutuhan laporan bulanan/tahunan.
- Pencatatan log aktivitas (activity log) setiap kali dokumen ditambah, diubah, atau dihapus, agar histori pengelolaan arsip dapat ditelusuri.
- Penerapan role dan permission (misalnya Admin, Petugas Arsip, dan Pimpinan) menggunakan package seperti Spatie Laravel-Permission, sehingga hak akses setiap fitur dapat diatur lebih rinci.
- Pencarian yang lebih canggih (full text search) menggunakan Laravel Scout apabila jumlah data arsip sudah sangat besar.
- Dukungan multi-file dalam satu dokumen arsip, dengan membuat tabel relasi tambahan seperti dokumen_lampiran.
- Notifikasi email atau sistem ketika ada dokumen baru yang diarsipkan pada kategori tertentu.

10.3 Referensi

- Dokumentasi resmi Laravel — <https://laravel.com/docs>
- Struktur alur belajar pada modul ini terinspirasi dari rangkaian Tutorial Laravel 13 untuk Pemula, SantriKoding.com — <https://santrikoding.com/tutorial-set/tutorial-laravel-13-untuk-pemula>

CATATAN PENTING

Modul ini disusun ulang dan disesuaikan sepenuhnya (kode, struktur database, dan studi kasus) untuk kebutuhan pembuatan Sistem Arsip Dokumen, dan tidak menyalin isi tulisan dari sumber referensi di atas. Sumber referensi dicantumkan semata-mata sebagai penghormatan atas inspirasi alur belajar CRUD Laravel yang digunakan.